

MLOps (S1-25_AIMLCZG523)

ASSIGNMENT – I

Objective :

Design, develop, and deploy a scalable and reproducible machine learning solution utilising modern MLOps best practices. The assignment emphasises practical automation, experiment tracking, CI/CD pipelines, containerization, cloud deployment, and monitoring—mirroring real-world production scenarios.

Dataset : Heart Disease UCI Dataset

(<https://archive.ics.uci.edu/ml/datasets/heart+Disease>)

Group : 23

Members & Contributions:

1. **Ashim Das (2023AC05090) – 100%**
2. Swayam Prakash Pattanayak (2024AA05995) – 0%
3. Thakkar Manan (2024AB05297) – 0%
4. **Thillaikkarasan M (2024AA05955) – 100 %**
5. V Sai Akshith (2024AA05866) – 0%

1. GitHub Repo Link :

https://github.com/ashimdas94/MLOps_Assignment_1

2. Setup/install instructions :

Prerequisites

1. Python 3.10+
2. pip
3. Docker
4. Kubectl
5. Minikube
6. Helm (only required for installing Prometheus / Grafana)

Installation

1 Clone the repository and install dependencies :

```
git clone https://github.com/ashimdas94/MLOps_Assignment_1.git
cd MLOps_Assignment_1
```

```
pip install -r requirements.txt
```

2 Download dataset :

```
python -m data.download_dataset
```

3 EDA :

```
jupyter notebook notebooks/eda.ipynb
```

4 Train model:

```
python -m src.train
```

5 Testing:

```
pytest tests \
--cov=src \
--cov-report=term \
--cov-report=xml \
--cov-report=html \
-v
```

6 Run API locally:

```
uvicorn api.main:app --reload --port 8000
```

7 Build Docker image:

```
docker build -f api/Dockerfile -t heart-api:latest .
```

8 Run via Docker:

```
docker run -p 8000:8000 heart-api:latest
```

Deployment Using Kubernetes (Minikube)

This project supports local Kubernetes deployment using **Minikube**, simulating a production-like environment with container orchestration, scaling, and monitoring.

1 Start Minikube

```
minikube start --driver=docker
```

Configure your shell to use Minikube's Docker daemon:

```
eval $(minikube docker-env)
```

2 Build Docker Image Inside Minikube

```
docker build -f api/Dockerfile -t heart-api:latest .
```

3 Deploy Application to Kubernetes

```
kubectl apply -f k8s/deployment.yaml  
kubectl apply -f k8s/service.yaml
```

Verify:

```
kubectl get pods  
kubectl get svc
```

4 Access the API

Stable local access (Forward to port 8000):

```
kubectl port-forward svc/heart-disease-predictor-service 8000:80
```

API available at:

<http://localhost:8000/predict>

5 Running the UI

```
streamlit run ui/app.py
```

Open in Browser : <http://localhost:8501/>

Monitoring with Prometheus & Grafana

The API exposes Prometheus-compatible metrics at /metrics and is monitored using **Prometheus Operator + Grafana**.

1 Enable Metrics Scraping

```
kubectl apply -f k8s/prometheus-config.yaml
```

Verify targets:

```
kubectl port-forward -n monitoring svc/prometheus-kube-prometheus-  
prometheus 9090
```

Open in browser : <http://localhost:9090>

3 Access Grafana

```
kubectl port-forward -n monitoring svc/prometheus-grafana 3000:80
```

Open <http://localhost:3000>

3. Exploratory Data Analysis (EDA)

Dataset Overview

The dataset used for this project is derived from the UCI Heart Disease dataset and consists of **303 patient records** with **14 attributes**, including demographic information, clinical measurements, and diagnostic indicators. The target variable represents the presence or absence of heart disease.

The dataset includes:

- **Numerical features** such as age, resting blood pressure (trestbps), cholesterol (chol), maximum heart rate achieved (thalach), and ST depression (oldpeak).
- **Categorical features** such as sex, chest pain type (cp), fasting blood sugar (fbs), resting ECG results (restecg), exercise-induced angina (exang), slope of ST segment (slope), number of major vessels (ca), and thalassemia (thal).

Data Quality & Missing Values

Initial inspection of the dataset showed that most features were complete; however:

- The feature **ca** had 4 missing values.
- The feature **thal** had 2 missing values.

These missing values originated from placeholder symbols in the raw dataset and were handled by **dropping affected rows during preprocessing**, as implemented in the preprocessing pipeline. Given the small proportion of missing records, this approach did not materially affect the dataset size or distribution.

Target Variable Analysis

The original target variable contains multiple values (1–4) indicating the presence of heart disease. For analysis and modeling, the target was **binarized**:

- 0 → Absence of heart disease
- 1 → Presence of heart disease

The resulting class distribution was:

- **54.13%** absence of disease
- **45.87%** presence of disease

This indicates that the dataset is **relatively balanced**, reducing the need for specialized imbalance handling techniques such as resampling or class weighting.

Feature Distribution Analysis

Histograms of numerical features revealed:

- **Age** follows an approximately normal distribution centered around middle-aged individuals.
- **Cholesterol** and **resting blood pressure** exhibit moderate spread with some right skew.
- **Maximum heart rate (thalach)** shows a wide range, suggesting strong discriminative potential.
- **ST depression (oldpeak)** is right-skewed, with most values concentrated at lower levels.

Rows with missing values were removed prior to visualization to ensure consistency with the preprocessing logic used during model training.

Correlation Analysis

A Pearson correlation heatmap was used to analyze relationships between features and the binarized target variable.

Strongest Positive Correlations with Heart Disease

The following features exhibited the highest positive correlation with disease presence:

- **Thalassemia (thal)**: $r \approx 0.53$
- **Number of major vessels (ca)**: $r \approx 0.46$
- **ST depression (oldpeak)**: $r \approx 0.42$
- **Exercise-induced angina (exang)**: $r \approx 0.42$
- **Chest pain type (cp)**: $r \approx 0.41$

These features are clinically intuitive and align with known cardiovascular risk indicators.

Strongest Negative Correlations

- **Maximum heart rate (thalach)** showed a strong negative correlation ($r \approx -0.42$), indicating that lower exercise tolerance is associated with higher likelihood of heart disease.

Weak or Negligible Correlations

- **Fasting blood sugar (fbs)** showed near-zero correlation with the target.
- **Resting ECG (restecg)** and **resting blood pressure (restbtps)** showed weak positive correlations.

Feature Interdependencies

Moderate correlations between features were observed (e.g., age vs. thalach, oldpeak vs. slope), but no extreme multicollinearity was detected, allowing all features to be retained for modelling.

EDA Summary

- The dataset is clean, structured, and well-suited for supervised learning.
- The target variable is reasonably balanced.
- Several features show meaningful correlation with heart disease presence.
- No extensive feature engineering or dimensionality reduction was required.

4. Modelling Choices

Problem Formulation

The task was formulated as a **binary classification problem** to predict the presence or absence of heart disease based on clinical features.

Feature Selection

All available features were retained for modelling due to:

- Their clinical relevance
- Absence of strong multicollinearity
- Observed correlation with the target variable

Features identified during EDA (e.g., thal, ca, cp, exang, thalach) were expected to contribute significantly to model performance.

Preprocessing Strategy

- Rows with missing values in ca and thal were removed.
- Numerical features were scaled where required.
- Categorical features were already encoded numerically, enabling direct use in classical machine learning models.

The preprocessing logic was centralized and reused consistently across training, inference, and testing pipelines.

Model Selection Rationale

Classical machine learning models such as **Logistic Regression** and **Random Forest** were chosen because:

- The dataset is relatively small and tabular.
- Interpretability is important for healthcare-related predictions.
- These models perform well without extensive feature engineering.
- They provide probability outputs, which are useful for monitoring and UI display.

Training & Experiment Tracking

- The dataset was split into training and test sets to assess generalization.

- Model training and evaluation were tracked using **MLflow**, logging parameters, metrics, and trained model artifacts.
- This enabled reproducibility, comparison of experiments, and seamless deployment.

5. Experiment Tracking Summary

Experiment tracking was implemented using **MLflow** to manage and compare multiple model training runs in a reproducible manner. All experiments were logged under a dedicated MLflow experiment, enabling systematic evaluation of different modelling approaches.

Tracked Experiments

Two primary models were trained and tracked:

1. Logistic Regression

- **Parameters logged:**
 - solver = liblinear
 - random_state = 42
- **Evaluation metrics logged:**
 - Accuracy: ~0.82
 - Precision: ~0.87
 - Recall: ~0.71
 - ROC-AUC: ~0.94
- The trained model was registered in the MLflow Model Registry as **Logistic_Regression_Pipeline (v1)**.

This model provided a strong baseline with high ROC-AUC and good interpretability, making it suitable for healthcare-related predictions.

2. Random Forest Classifier

- **Parameters logged:**
 - n_estimators = 100
 - max_depth = 5
 - random_state = 42
- **Evaluation metrics logged:**
 - Accuracy: ~0.87
 - Precision: ~0.92
 - Recall: ~0.79
 - ROC-AUC: ~0.95
- The trained model was registered as **Random_Forest_Pipeline (v1)**.

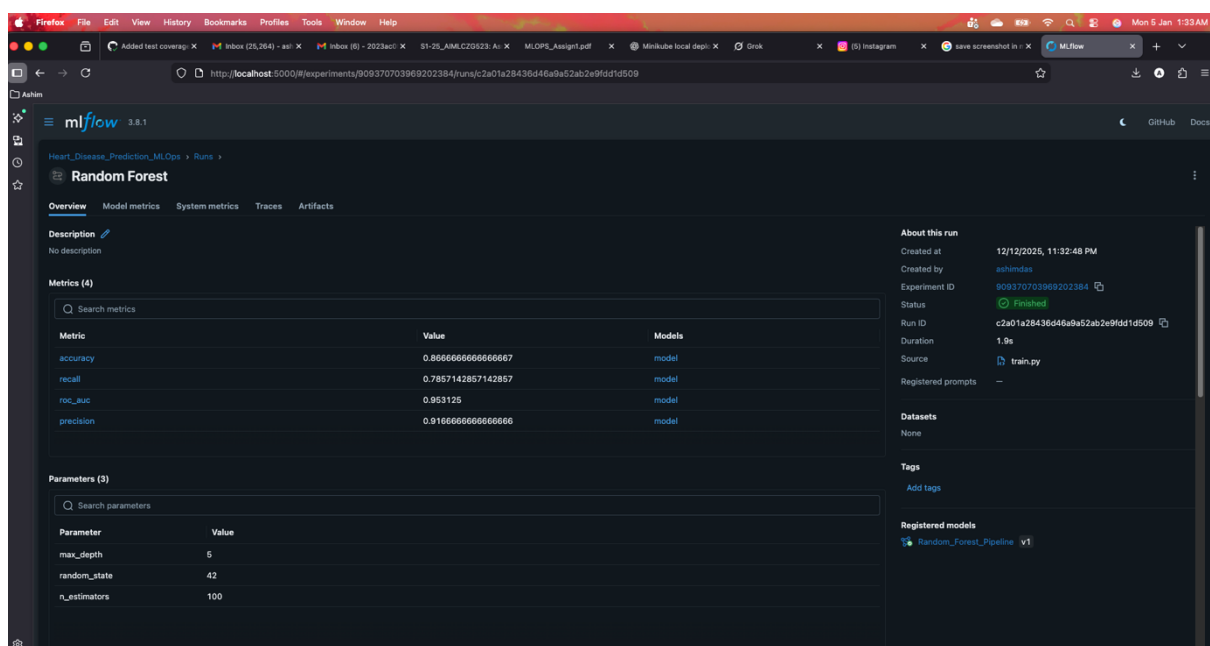
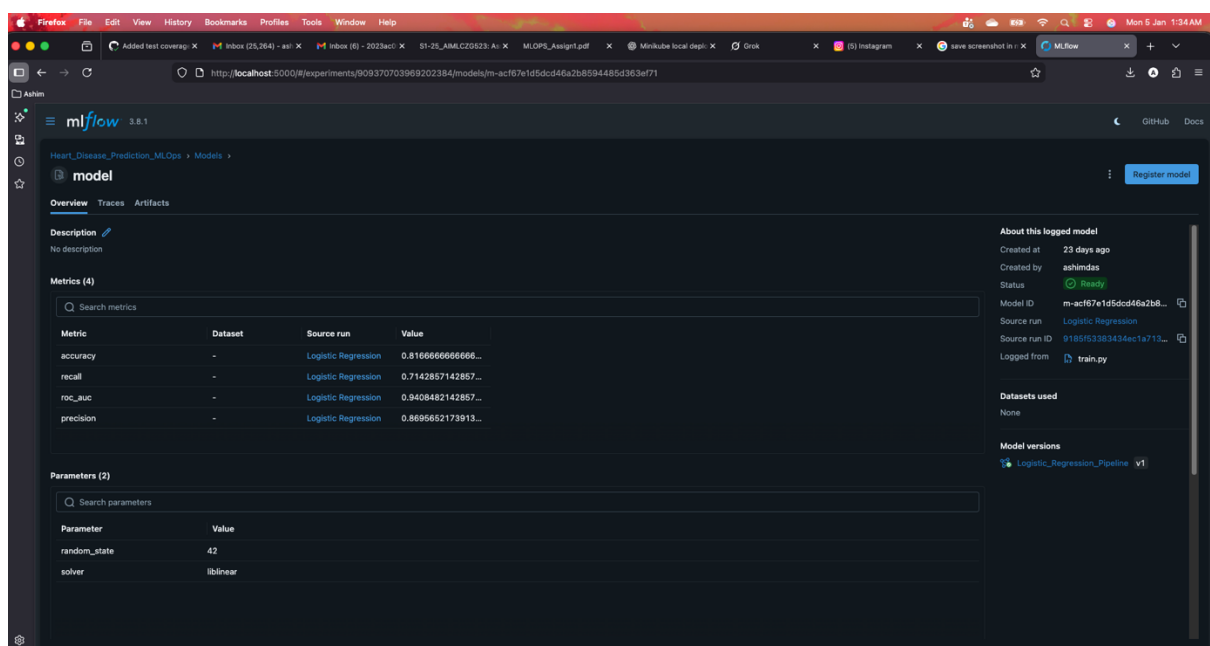
Compared to Logistic Regression, the Random Forest model achieved higher accuracy, recall, and precision, indicating improved classification performance while maintaining robust generalization.

Model Comparison & Selection

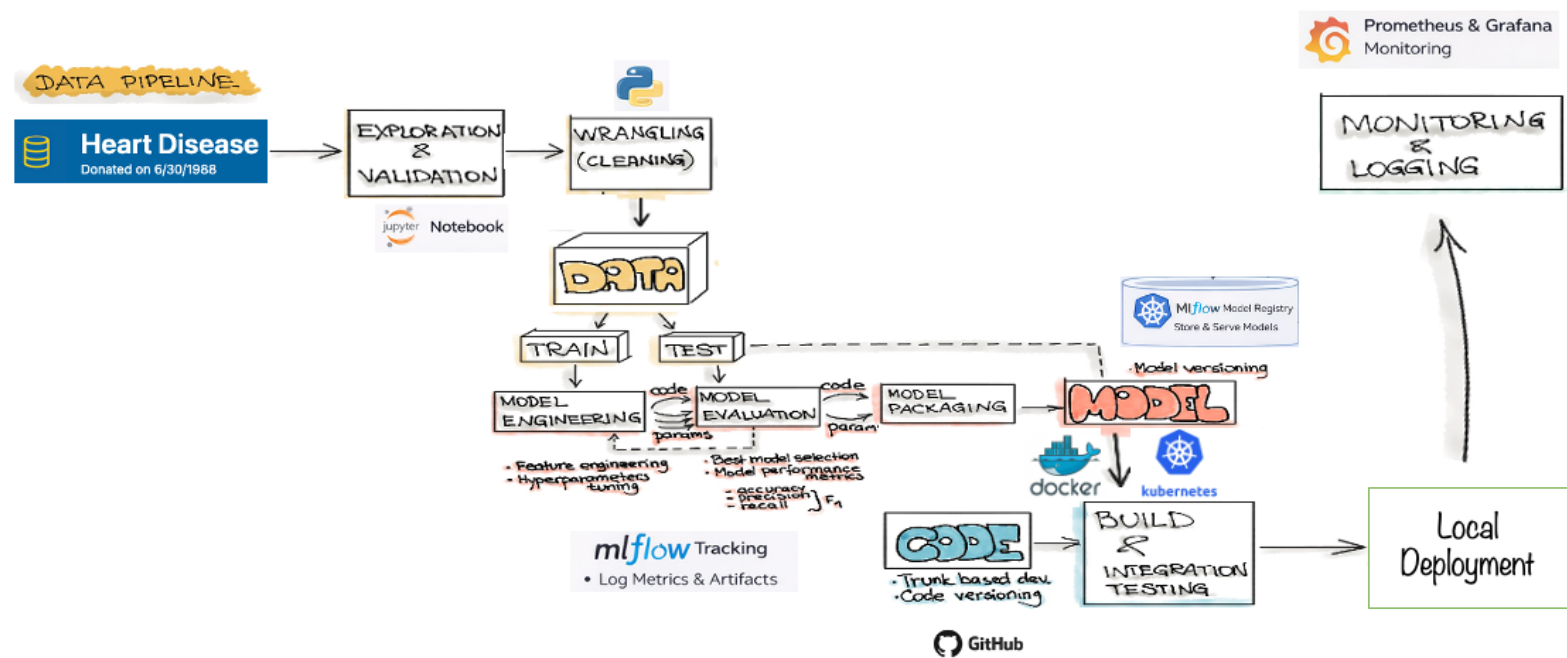
MLflow enabled direct comparison between the two models using standardized metrics logged during training. Based on the tracked results:

- The **Random Forest model** demonstrated superior performance across most evaluation metrics.
- Both models were preserved and versioned, allowing traceability and future comparison.
- The selected model for deployment can be traced back to its exact training configuration and performance metrics.

MLflow UI screenshots :



6. Architecture Diagram



7. CI/CD and deployment workflow screenshots

GitHub Actions Workflow

This screenshot shows the details of a GitHub Actions workflow run named 'build-and-test'. The workflow is part of the 'MLops CI/CD Pipeline' and has successfully completed. The left sidebar shows the 'Summary' tab selected, with a list of jobs including 'build-and-test'. The main area displays a list of steps in the workflow, each with a status icon and a duration. The steps are: Set up job (1s), Checkout Repository (1s), Set up Python (0s), Install Dependencies (44s), Lint code (1s), Download Data (2s), Train Model and Log with MLflow (Mock) (0s), Run tests with coverage (3s), Upload coverage report (0s), Upload trained model (1s), Upload MLflow runs (1s), Upload CI logs (0s), Cleanup Container (0s), Post Set up Python (1s), Post Checkout Repository (0s), and Complete job (0s).

Step	Duration
Set up job	1s
Checkout Repository	1s
Set up Python	0s
Install Dependencies	44s
Lint code	1s
Download Data	2s
Train Model and Log with MLflow (Mock)	0s
Run tests with coverage	3s
Upload coverage report	0s
Upload trained model	1s
Upload MLflow runs	1s
Upload CI logs	0s
Cleanup Container	0s
Post Set up Python	1s
Post Checkout Repository	0s
Complete job	0s

Artifacts

This screenshot shows the 'Artifacts' section of the GitHub Actions workflow run. It displays a table of artifacts produced during the runtime, including their names, sizes, and digests. The artifacts are: ci-logs (1.21 KB), mlflow-runs (74.5 KB), test-coverage (39.4 KB), and trained-model (69.2 KB). The table also includes a 'Name' column, a 'Size' column, and a 'Digest' column. The artifacts are listed in a table with columns for Name, Size, and Digest. Each row includes a download icon and a trash icon.

Name	Size	Digest
ci-logs	1.21 KB	sha256:b8d8ee77d7f8302691d5e44dc81699de2989419531f3464a62b188...
mlflow-runs	74.5 KB	sha256:9ea250754a76b3287d93f31e9cf414bf6c2d80f24dbbc767f76cb7...
test-coverage	39.4 KB	sha256:de810d2abe9c3935ec69ece5cf605d55af8bde881eb33b16ccb1f9...
trained-model	69.2 KB	sha256:84661aa7960999ba9b87bc6e19daf851c2f8867d70097aa34832e1...

Deployment – Kubernetes Pods

```
Terminal Local × Local (2) × Local (3) × Local (4) × Local (5) × + ∨

(ai_dev_1) ashimdas@MacBookAir MLOps_Assignment_1 % kubectl get pods

NAME                                READY   STATUS    RESTARTS   AGE
heart-disease-predictor-deployment-644b89f697-2g2gh  1/1     Running   1 (3m24s ago)  10d
heart-disease-predictor-deployment-644b89f697-ks5jl  1/1     Running   1 (3m24s ago)  10d

(ai_dev_1) ashimdas@MacBookAir MLOps_Assignment_1 % kubectl get svc
NAME                                TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)          AGE
heart-disease-predictor-service     NodePort    10.102.54.27 <none>        80:32190/TCP     10d
kubernetes                          ClusterIP   10.96.0.1    <none>        443/TCP          10d

(ai_dev_1) ashimdas@MacBookAir MLOps_Assignment_1 % kubectl get pods -n monitoring
NAME                                READY   STATUS    RESTARTS   AGE
alertmanager-prometheus-kube-prometheus-alertmanager-0  2/2     Running   2 (8m29s ago)  10d
prometheus-grafana-6dc567459d-zcwrz  3/3     Running   3 (8m29s ago)  10d
prometheus-kube-prometheus-operator-6b9b555fd9-7k6h6  1/1     Running   1 (8m29s ago)  10d
prometheus-kube-state-metrics-66f9f5bf55-jc5vk  1/1     Running   1 (8m29s ago)  10d
prometheus-prometheus-kube-prometheus-prometheus-0  2/2     Running   2 (8m29s ago)  10d
prometheus-prometheus-node-exporter-62hwn  1/1     Running   1 (8m29s ago)  10d
```

Minikube Service Execution

```
(ai_dev_1) ashimdas@MacBookAir MLOps_Assignment_1 % minikube service heart-disease-predictor-service
```

NAMESPACE	NAME	TARGET PORT	URL
default	heart-disease-predictor-service	http/80	http://192.168.49.2:32190

```
✎ Starting tunnel for service heart-disease-predictor-service. /
NAMESPACE NAME TARGET PORT URL
default heart-disease-predictor-service http://127.0.0.1:58505

✎ Starting tunnel for service heart-disease-predictor-service.
✎ Opening service default/heart-disease-predictor-service in default browser...
! Because you are using a Docker driver on darwin, the terminal needs to be open to run it.
```

Heart API Prediction (Negative Result)

Heart Disease Prediction

Enter patient details to get a real-time prediction from the FastAPI service deployed on Kubernetes.

Patient Details

Age

50

Sex (0 = Female, 1 = Male)

0

Chest Pain Type (0-3)

1

Resting Blood Pressure

120

Cholesterol (mg/dl)

200

Fasting Blood Sugar > 120 mg/dl

0

Resting ECG (0-2)

0

Max Heart Rate

150

Exercise Induced Angina

0

ST Depression

1.00

Slope (0-2)

1

Major Vessels (0-3)

0

Thalassemia (1-3)

2

Predict

Prediction Result

No Heart Disease Detected

Message: No heart disease predicted

Prediction Probability

0.15

Latency: 88.53 ms

Heart API Prediction (Positive Result)

Heart Disease Prediction

Enter patient details to get a real-time prediction from the FastAPI service deployed on Kubernetes.

Patient Details

Age

60

Sex (0 = Female, 1 = Male)

1

Chest Pain Type (0-3)

2

Resting Blood Pressure

140

Cholesterol (mg/dl)

240

Fasting Blood Sugar > 120 mg/dl

0

Resting ECG (0-2)

0

Max Heart Rate

160

Exercise Induced Angina

1

ST Depression

1.40

Slope (0-2)

1

Major Vessels (0-3)

1

Thalassemia (1-3)

2

Predict

Prediction Result

Heart Disease Detected

Message: Heart disease predicted

Prediction Probability

0.53

Latency: 98.62 ms